Object Oriented Programming Language

Object-Oriented Thinking

Ji-Hyeong Han

(jhhan@seoultech.ac.kr)

Dept. of Computer Science and Engineering

SEOULTECH

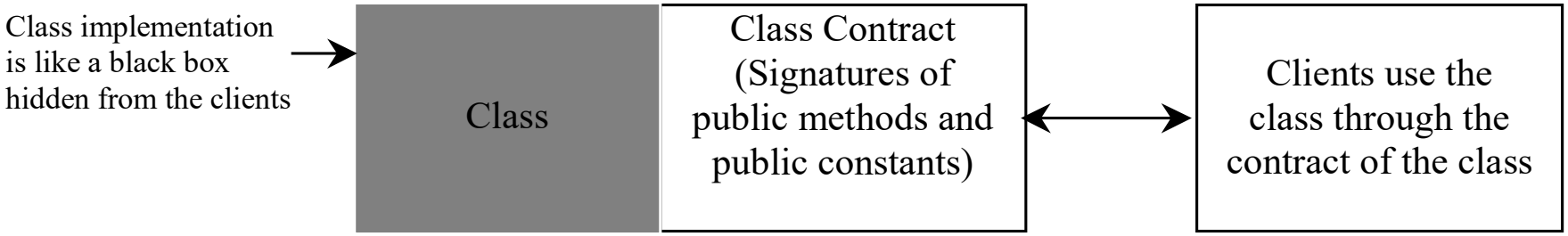
Motivations

- You see the advantages of object-oriented programming from the preceding chapter.

- This chapter will demonstrate how to solve problems using the object-oriented paradigm.

Class Abstraction and Encapsulation

- Class abstraction
  - Separate class implementation from the use of the class
  - The creator of the class provides a description of the class and let the user know how the class can be used
  - The user of the class does not need to know how the class is implemented
  - The detail of implementation is encapsulated and hidden from the user.



Example: Designing the Loan Class

Loan	
-annualInterestRate: double	The annual interest rate of the loan (default: 2.5).
-numberOfYears: int	The number of years for the loan (default: 1)
-loanAmount: double	The loan amount (default: 1000).
-loanDate: Date	The date this loan was created.
+Loan()	Constructs a default Loan object.
+Loan(annualInterestRate: double, numberOfYears: int, loanAmount: double)	Constructs a loan with specified interest rate, years, and loan amount.
+getAnnualInterestRate(): double	Returns the annual interest rate of this loan.
+getNumberOfYears(): int	Returns the number of the years of this loan.
+getLoanAmount(): double	Returns the amount of this loan.
+getLoanDate(): Date	Returns the date of the creation of this loan.
+setAnnualInterestRate(annualInterestRate: double): void	Sets a new annual interest rate to this loan.
+setNumberOfYears(numberOfYears: int): void	Sets a new number of years to this loan.
+setLoanAmount(loanAmount: double): void	Sets a new amount to this loan.
+getMonthlyPayment(): double	Returns the monthly payment of this loan.
+getTotalPayment(): double	Returns the total payment of this loan.

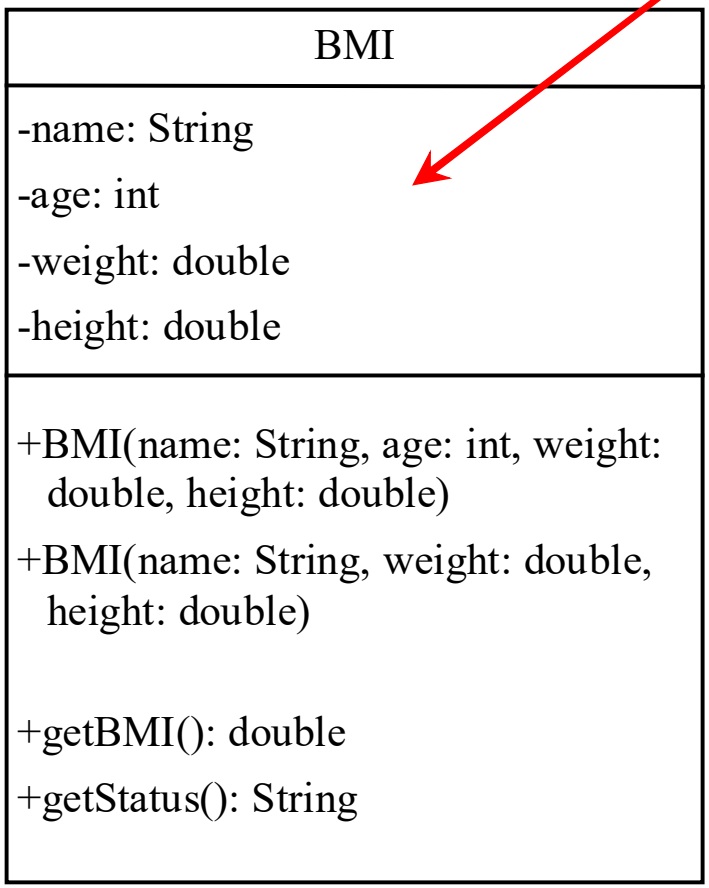
Loan

TestLoanClass

Object-Oriented Thinking

- The procedural paradigm focuses on designing methods
- The object-oriented paradigm couples data and methods together into objects
- Classes provide more flexibility and modularity for building reusable software.
- From the improvements, you will gain the insight on the differences between the procedural programming and object-oriented programming and see the benefits of developing reusable code using objects and classes.

Example: The BMI Class



The get methods for these data fields are provided in the class, but omitted in the UML diagram for brevity.

The name of the person.

The age of the person.

The weight of the person in pounds.

The height of the person in inches.

Creates a BMI object with the specified name, age, weight, and height.

Creates a BMI object with the specified name, weight, height, and a default age 20.

Returns the BMI

Returns the BMI status (e.g., normal, overweight, etc.)

BMI

UseBMIClass

Class Relationship

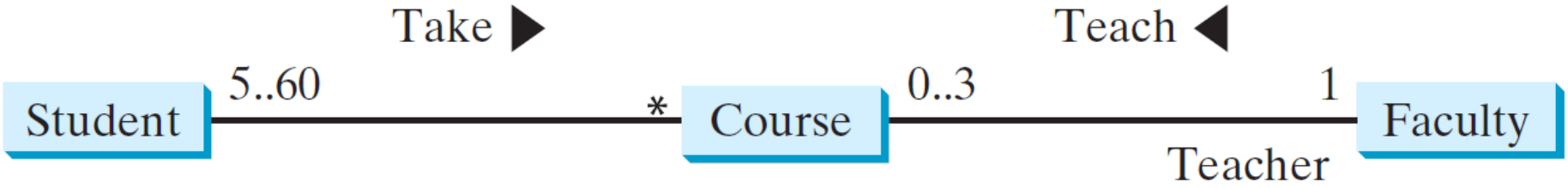
- To design classes, you need to explore the relationship among classes

- The common relations among classes
  - Association
  - Aggregation
  - Composition
  - Inheritance (will be introduced in the next chapter)

Class Relationship - Association

- A general binary relationship that describes an activity between two classes

- Example




```
public class Student {
  private Course[]
    courseList;

  public void addCourse(
    Course s) { ... }
}
```

```
public class Course {
  private Student[]
    classList;
  private Faculty faculty;

  public void addStudent(
    Student s) { ... }

  public void setFaculty(
    Faculty faculty) { ... }
}
```

```
public class Faculty {
  private Course[]
    courseList;

  public void addCourse(
    Course c) { ... }
}
```

Class Relationship – Aggregation and Composition

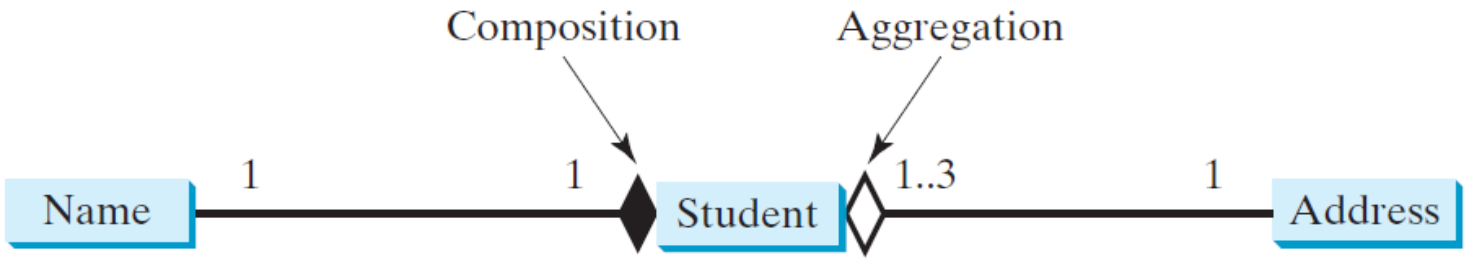
- Aggregation is a special form of association
  - Represents an ownership relationship between two objects
  - *has-a* relationships
  - The owner object is called an *aggregating object* and its class an *aggregating class*
  - The subject object is called an *aggregated object* and its class an *aggregated class*

- Composition is a special case of the aggregation relationship
  - When an object is exclusively owned by an aggregating object, the relationship between the object and its aggregating object is referred to as a composition

- Example
  - Composition: “a student has a name” (Student class and Name class)
  - Aggregation: “a student has an address” (Student class and Address class, since an address can be shared by several students)

Class Relationship – Aggregation and Composition

- Example




```
public class Name {
  ...
}
```

Aggregated class

```
public class Student {
  private Name name;
  private Address address;
  ...
}
```

Aggregating class

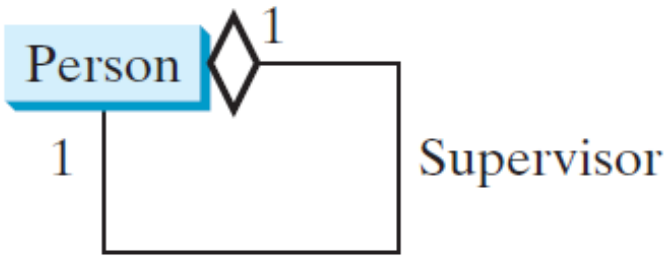
```
public class Address {
  ...
}
```

Aggregated class

- An aggregation relationship is usually represented as a data filed in the aggregating class

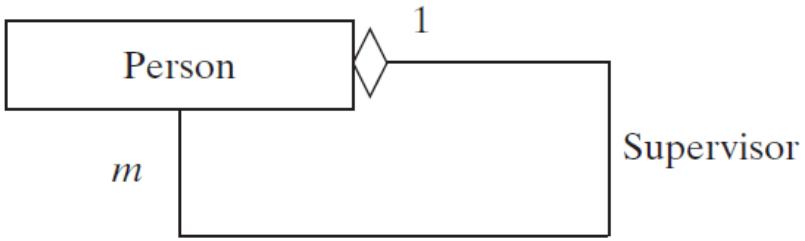
Class Relationship – Aggregation Between Same Class

- Aggregation may exist between objects of the same class
- For example, a person may have a supervisor




```
public class Person {
  // The type for the data is the class itself
  private Person supervisor;
  ...
}
```

- What happens if a person has several supervisor?




```
public class Person {
  ...
  private Person[] supervisors;
}
```

Example: The Course Class

Course	
-courseName: String	The name of the course.
-students: String[]	An array to store the students for the course.
-numberOfStudents: int	The number of students (default: 0).
+Course(courseName: String)	Creates a course with the specified name.
+getCourseName(): String	Returns the course name.
+addStudent(student: String): void	Adds a new student to the course.
+dropStudent(student: String): void	Drops a student from the course.
+getStudents(): String[]	Returns the students in the course.
+getNumberOfStudents(): int	Returns the number of students in the course.

Course

TestCourse

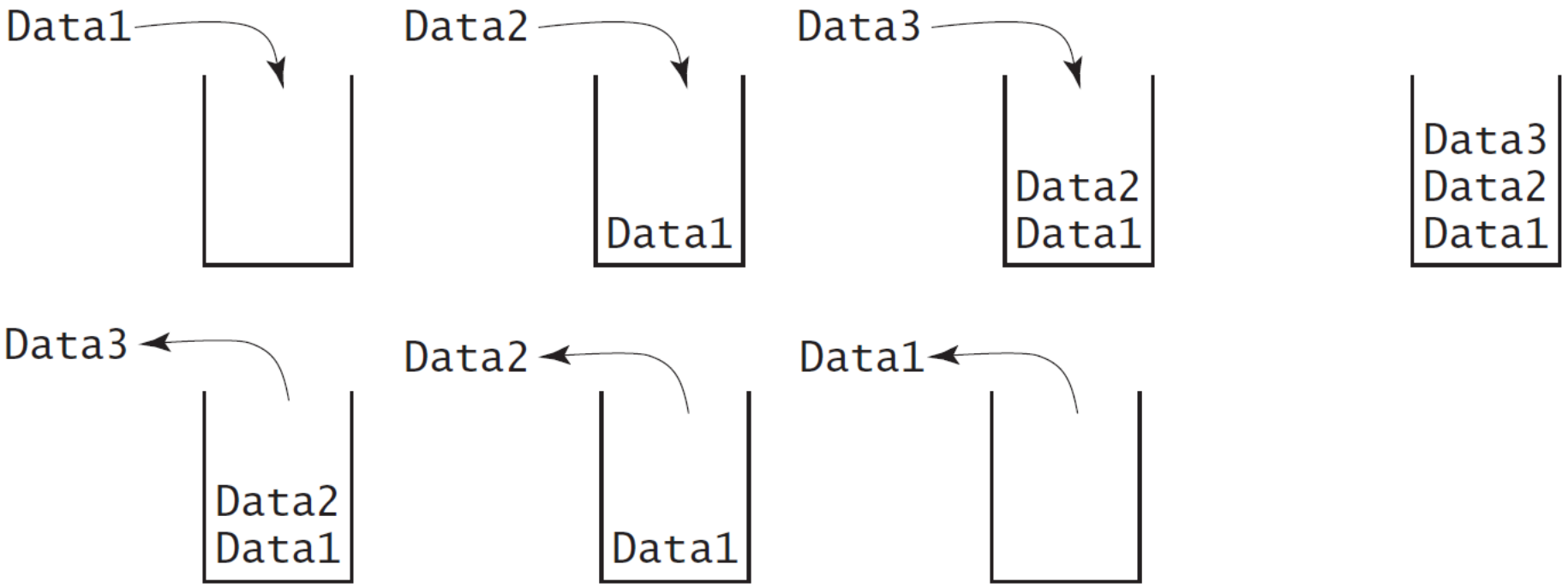
Example: The `StackOfIntegers` Class

StackOfIntegers	
-elements: int[]	An array to store integers in the stack.
-size: int	The number of integers in the stack.
+StackOfIntegers()	Constructs an empty stack with a default capacity of 16.
+StackOfIntegers(capacity: int)	Constructs an empty stack with a specified capacity.
+empty(): boolean	Returns true if the stack is empty.
+peek(): int	Returns the integer at the top of the stack without removing it from the stack.
+push(value: int): int	Stores an integer into the top of the stack.
+pop(): int	Removes the integer at the top of the stack and returns it.
+getSize(): int	Returns the number of elements in the stack.

TestStackOfIntegers

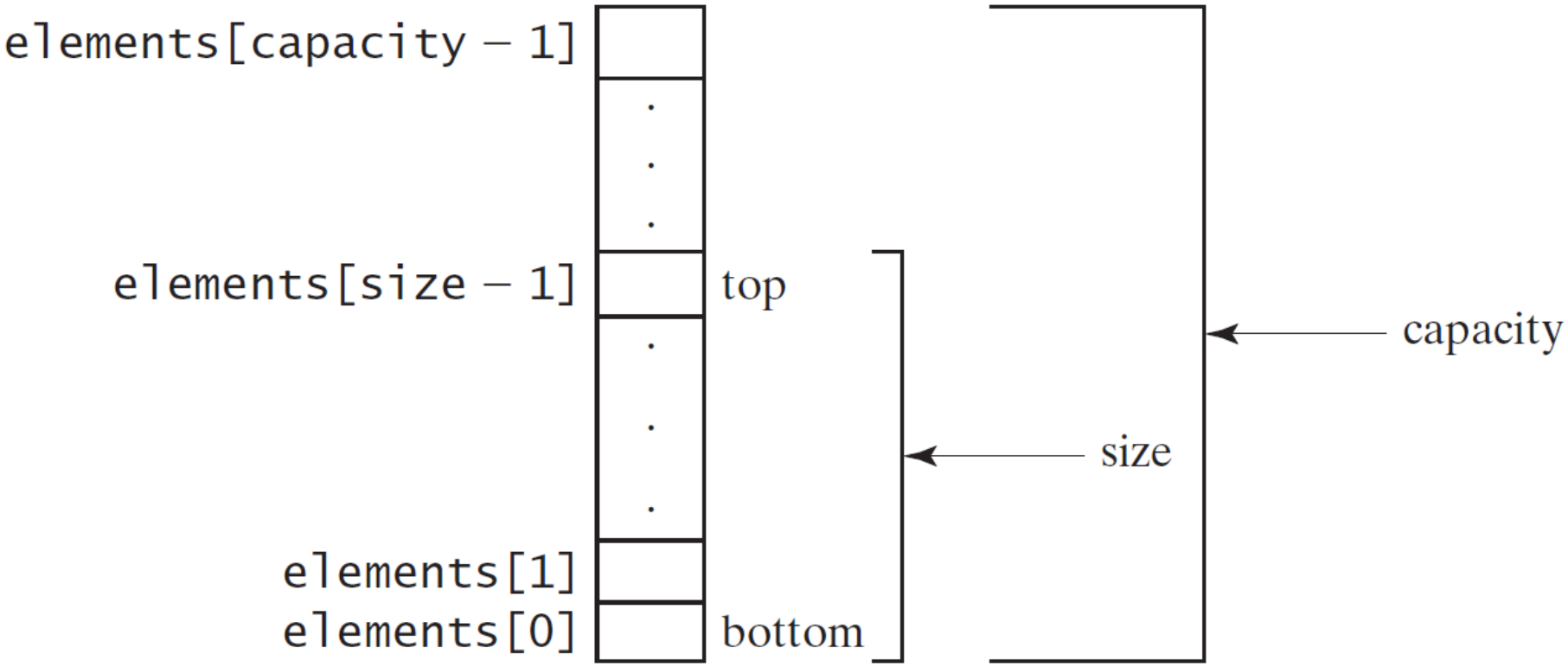
Example: The `StackOfIntegers` Class

- Designing the StackOfIntegers Class
  - Stack: last-in, first-out



Example: The `StackOfIntegers` Class

- Implementing `StackOfIntegers` Class



Wrapper Classes

- A primitive type value is not an object, but it can be wrapped in an object using a wrapper class in the JAVA API (java.lang package)
  - However, many Java methods require the use of objects as argument
  - So, Java offers a convenient way to wrap a primitive data type into an object
  - e.g. wrapping int into the Integer class, wrapping double into the Double class

- Boolean
- Character
- Short
- Byte
- Integer
- Long
- Float
- Double

NOTE: (1) The wrapper classes do not have no-arg constructors. (2) The instances of all wrapper classes are immutable, i.e., their internal values cannot be changed once the objects are created.

The `Integer` and `Double` Classes

java.lang.Integer
-value: int +MAX_VALUE: int +MIN_VALUE: int
+Integer(value: int) +Integer(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longVlaue(): long +floatValue(): float +doubleValue():double +compareTo(o: Integer): int +toString(): String +valueOf(s: String): Integer +valueOf(s: String, radix: int): Integer +parseInt(s: String): int +parseInt(s: String, radix: int): int

java.lang.Double
-value: double +MAX_VALUE: double +MIN_VALUE: double
+Double(value: double) +Double(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longVlaue(): long +floatValue(): float +doubleValue():double +compareTo(o: Double): int +toString(): String +valueOf(s: String): Double +valueOf(s: String, radix: int): Double +parseDouble(s: String): double +parseDouble(s: String, radix: int): double

The `Integer` and `Double` Classes

- Constructor
  - You can construct a wrapper object either from a primitive data type value or from a string representing the numeric value
  - Example

    new Double(5.0), new Double("5.0")
    new Integer(5), new Integer("5")

- Class Constants `MAX_VALUE`, `MIN_VALUE`

- Conversion Methods
  - doubleValue(), floatValue(), intValue(), longValue(), shortValue()
  - For example

    new Double(12.4).intValue() returns 12
    new Integer(12).doubleValue() returns 12.0

The `Integer` and `Double` Classes

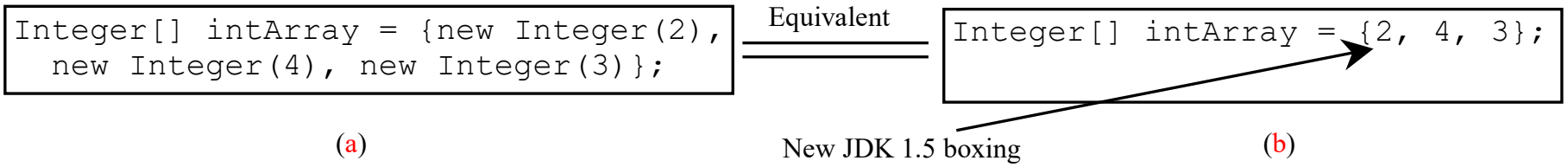
- The Static valueOf Methods
  - The numeric wrapper classes have a useful class method, valueOf(String s)
  - This method creates a new object initialized to the value represented by the specified string. For example:

```
Double doubleObject = Double.valueOf("12.4");
Integer integerObject = Integer.valueOf("12");
```

- The Methods for Parsing Strings into Numbers
  - You have used the parseInt method in the Integer class to parse a numeric string into an int value and the parseDouble method in the Double class to parse a numeric string into a double value
  - Each numeric wrapper class has two overloaded parsing methods to parse a numeric string into an appropriate numeric value.

Automatic Conversion Between Primitive Types and Wrapper Class Types

- JDK 1.5 allows primitive type and wrapper classes to be converted automatically.
- For example, the following statement in (a) can be simplified as in (b):




```
Integer[] intArray = {1, 2, 3};
System.out.println(intArray[0] + intArray[1] + intArray[2]);
```

Unboxing

BigInteger and BigDecimal

- If you need to compute with very large integers or high precision floating-point values, you can use the BigInteger and BigDecimal classes in the java.math package.

- Both are *immutable*. Both extend the Number class and implement the Comparable interface.

BigInteger and BigDecimal

```
BigInteger a = new BigInteger("92233720368547758O7");
BigInteger b = new BigInteger("2");
BigInteger c = a.multiply(b); // 92233720368547758O7 * 2
System.out.println(c);
```

LargeFactorial

```
BigDecimal a = new BigDecimal(1.0);
BigDecimal b = new BigDecimal(3);
BigDecimal c = a.divide(b, 20, BigDecimal.ROUND_UP);
System.out.println(c);
```

cf) The largest integer of the long type is 9223372036854775807

The `String` Class

- Constructing Strings

  String newString = new String(stringLiteral);
  String message = new String("Welcome to Java");

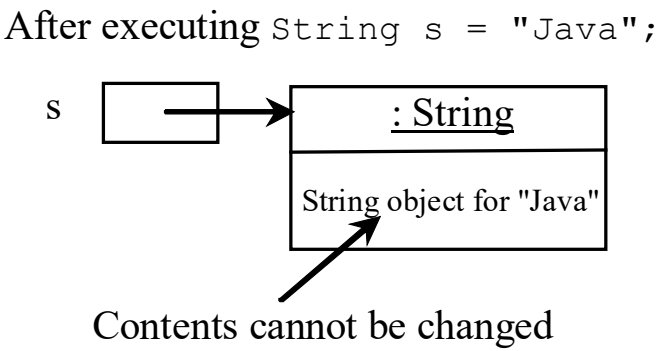
- Since strings are used frequently, Java provides a shorthand initializer for creating a string:

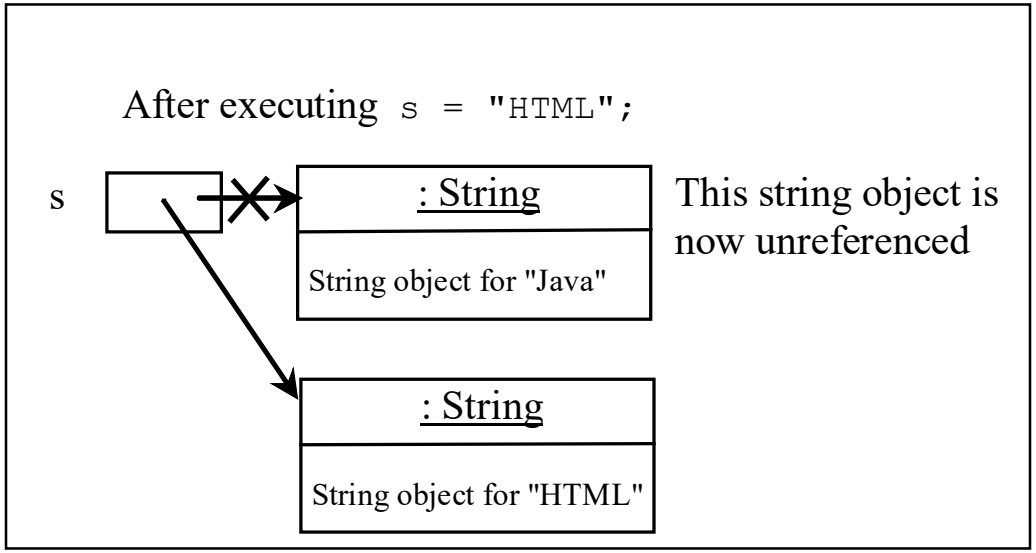
  String message = "Welcome to Java";

The `String` Class

- Strings Are Immutable
- A String object is immutable; its contents cannot be changed.
- Does the following code change the contents of the string?

```
String s = "Java";
s = "HTML";
```





The `String` Class

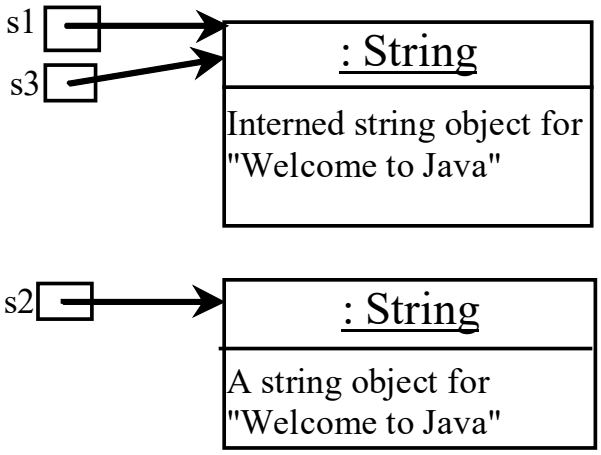
- Interned Strings
  - Since strings are immutable and are frequently used, to improve efficiency and save memory, the JVM uses a unique instance for string literals with the same character sequence.
  - Such an instance is called *interned*.

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";

System.out.println("s1 == s2 is " + (s1 == s2));
System.out.println("s1 == s3 is " + (s1 == s3));
```



display

s1 == s2 is false

s1 == s3 is true

A new object is created if you use the new operator.

If you use the string initializer, no new object is created if the interned object is already created.

The `String` Class

- Replacing and Splitting Strings

java.lang.String	
+replace(oldChar: char, newChar: char): String	Returns a new string that replaces all matching character in this string with the new character.
+replaceFirst(oldString: String, newString: String): String	Returns a new string that replaces the first matching substring in this string with the new substring.
+replaceAll(oldString: String, newString: String): String	Returns a new string that replace all matching substrings in this string with the new substring.
+split(delimiter: String): String[]	Returns an array of strings consisting of the substrings split by the delimiter.

"Welcome".replace('e', 'A') returns a new string, WAlcomA.

"Welcome".replaceFirst("e", "AB") returns a new string, WABlcome.

"Welcome".replace("e", "AB") returns a new string, WABlcomAB.

"Welcome".replace("el", "AB") returns a new string, WABcome.

The `String` Class

- Replacing and Splitting Strings

```
String[] tokens = "Java#HTML#Perl".split("#", 0);
for (int i = 0; i < tokens.length; i++)
  System.out.print(tokens[i] + " ");
```

displays

Java HTML Perl

The `String` Class

- Matching, Replacing and Splitting by Patterns

```
"Java".matches("Java");
"Java".equals("Java");
```

```
"Java is fun".matches("Java.*");
"Java is cool".matches("Java.*");
```

```
String[] tokens = "Java,C?C#,C++.split("[.,:;?]");
for (int i = 0; i < tokens.length; i++)
  System.out.println(tokens[i]);


Outut: Java
       C
       C#
       C++
```

The `String` Class

- Convert Character and Numbers to Strings
  - The String class provides several static valueOf methods for converting a character, an array of characters, and numeric values to strings.
  - These methods have the same name valueOf with different argument types char, char[], double, long, int, and float.
  - For example, to convert a double value to a string, use String.valueOf(5.44).
    The return value is string consists of characters ‘5’, ‘.’, ‘4’, and ‘4’.

The `StringBuilder` and `StringBuffer` Classes

- The `StringBuilder`/`StringBuffer` class is an alternative to the `String` class.
- In general, a StringBuilder/StringBuffer can be used wherever a string is used.
- StringBuilder/StringBuffer is more flexible than String.
- You can add, insert, or append new contents into a string buffer, whereas the value of a String object is fixed once the string is created.
- Difference between StringBuilder and StringBuffer
  - StringBuilder: no-sync in multithread
  - StringBuffer: sync in multithread

The `StringBuilder` and `StringBuffer` Classes

- `StringBuilder` Constructors

java.lang.StringBuilder	
+StringBuilder()	Constructs an empty string builder with capacity 16.
+StringBuilder(capacity: int)	Constructs a string builder with the specified capacity.
+StringBuilder(s: String)	Constructs a string builder with the specified string.

The `StringBuilder` and `StringBuffer` Classes

- Modifying Strings in the Builder

java.lang.StringBuilder	
+append(data: char[]): StringBuilder	Appends a char array into this string builder.
+append(data: char[], offset: int, len: int): StringBuilder	Appends a subarray in data into this string builder.
+append(v: *aPrimitiveType*): StringBuilder	Appends a primitive type value as a string to this builder.
+append(s: String): StringBuilder	Appends a string to this string builder.
+delete(startIndex: int, endIndex: int): StringBuilder	Deletes characters from startIndex to endIndex.
+deleteCharAt(index: int): StringBuilder	Deletes a character at the specified index.
+insert(index: int, data: char[], offset: int, len: int): StringBuilder	Inserts a subarray of the data in the array to the builder at the specified index.
+insert(offset: int, data: char[]): StringBuilder	Inserts data into this builder at the position offset.
+insert(offset: int, b: *aPrimitiveType*): StringBuilder	Inserts a value converted to a string into this builder.
+insert(offset: int, s: String): StringBuilder	Inserts a string into this builder at the position offset.
+replace(startIndex: int, endIndex: int, s: String): StringBuilder	Replaces the characters in this builder from startIndex to endIndex with the specified string.
+reverse(): StringBuilder	Reverses the characters in the builder.
+setCharAt(index: int, ch: char): void	Sets a new character at the specified index in this builder.

The `StringBuilder` and `StringBuffer` Classes

- Examples

```
StringBuilder stringBuilder = new StringBuilder();
stringBuilder.append("Welcome");
stringBuilder.append(' ');
stringBuilder.append("to");
stringBuilder.append(' ');
stringBuilder.append("Java");
```

stringBuilder.insert(11, "HTML and "); changes the builder to Welcome to HTML and Java.

stringBuilder.delete(8, 11) changes the builder to Welcome Java.

stringBuilder.deleteCharAt(8) changes the builder to Welcome o Java.

stringBuilder.reverse() changes the builder to avaJ ot emocleW.

stringBuilder.replace(11, 15, "HTML")
changes the builder to Welcome to HTML.

stringBuilder.setCharAt(0, 'w') sets the builder to welcome to Java.

Problem: Checking Palindromes Ignoring Non-alphanumeric Characters

- This example gives a program that counts the number of occurrence of each letter in a string.

- Assume the letters are not case-sensitive.

PalindromeIgnoreNonAlphanumeric

Practice

Practice #1

- (The Course class) Revise the Course class as follows:
  - The array size is fixed in the provided Course class. Improve it to **automatically increase the array size** by creating a new larger array and copying the contents of the current array to it (like stack class)
  - Implement the dropStudent(String student) method
  - Add a new method named clear() that removes all students from the course
  - Write a test program that creates two courses, adds three and four students for each course, removes one student from each course, and displays the students in the course at each step

```
Number of students in course1: 3
Peter Jones, Brian Smith, Anne Kennedy,
Number of students in course2: 4
S1, S2, S3, S4,
Number of students in course1 after drop Peter Jones: 2
Brian Smith, Anne Kennedy,
Number of students in course1 after clear: 0
Number of students in course2 after drop S2: 3
S1, S3, S4,
Number of students in course2 after clear: 0
```

Course

Practice #2

StackOfIntegers

- Today's lecture gives a class for Stack.
  Design a class named Queue for storing integers.
  Like a stack, a queue holds elements.
  In a **stack**, the elements are retrieved in a **last-in first-out** fashion.
  In a **queue**, the elements are retrieved in a **first-in first-out** fashion.
  - An int[] data field named elements that stores the int values in the queue
  - A data filed named size that stores the number of elements in the queue
  - A constructor that creates a Queue object with default capacity 8
  - The method named enqueue(int v) that adds v into the queue
  - The method named dequeue() that removes and returns the element from the queue
  - The method named empty() that returns true if the queue is empty
  - The method named getSize() that returns the size of the queue
- The array size **will be doubled** once **the number of the elements exceeds the size**
- After an element is removed from the beginning of the array (=when calling dequeue()), you **need to shift all elements in the array one position the left**

Practice #2 (cont'd)

- Write a test program that adds 20 numbers from 1 to 20 into the queue and removes these numbers and displays them

```
public static void main(String[] args) {
    Queue queue = new Queue();
    for (int i = 1; i <= 20; i++)
      queue.enqueue(i);

    while (!queue.empty())
      System.out.print(queue.dequeue() + " ");
  }
```